Hitesh Rathore(HR) *Notes*

LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

① CSS SELECTORS — COMPLETE GUIDE

SELECTOR TYPE	SYNTAX / CODE	REAL WORLD INPUT (HTML)	BROWSER OUTPUT (RESULT)	EXPLANATION

① .class
CLASS SELECTOR
	`.first {`
	`color: cyan;`
	`}`
	`<p class="first">`
	`Hello World`
	`</p>`
		index.html
		`<p class="first">`
		`Hello World`
		`</p>`
			index.html
			Hello World
				Targets elements using class names

② #id
ID SELECTOR
	`#first {`
	`color: red;`
	`}`
	`<p id="first">`
	`Hello`
	`</p>`
		index.html
		`<p id="first">`
		`Hello`
		`</p>`
			index.html
			Hello
				Targets one unique element

③ h2
ELEMENT SELECTOR
	`h2 {`
	`color: green;`
	`}`
		index.html
		`<h2>Hello CSS</h2>`
			index.html
			Hello CSS
				Targets elements using tag names

④ *
UNIVERSAL SELECTOR
	`* {`
	`margin: 0;`
	`}`
		index.html
		`<h1>Hello</h1>`
		`<p>Paragraph</p>`
		`<div>Box</div>`
			index.html
			Hello
			Paragraph
			Box
				Selects ALL elements

⑤ GROUPING SELECTOR	`h1, h2, p {`
	`color: orange;`
	`}`
		REAL WORLD INPUT (HTML)
		index.html
		`<h1>Heading 1</h1>`
		`<h2>Heading 2</h2>`
		`<p>Paragraph text</p>`
			BROWSER OUTPUT (RESULT)
			index.html
			Heading 1
			Heading 2
			Paragraph text
				Apply the same style to multiple selectors

⑥ [ ]
ATTRIBUTE SELECTOR
	`p[draggable="true"] {`
	`color: yellow;`
	`}`
	`<p draggable="true">`
	`Drag me`
	`</p>`
		REAL WORLD INPUT (HTML)
		index.html
		`<p draggable="true">`
		`Drag me`
		`</p>`
			BROWSER OUTPUT (RESULT)
			index.html
			Drag me
				Targets elements using attributes

DOM RELATIONSHIPS — COMPLETE GUIDE

① PARENT

A parent is an element that directly contains another element.

EXAMPLE

```
<section>
  <p>Text</p>
</section>
```

Here, <section> is the parent of <p>.

Example in DOM

section is parent of p

div is parent of (second) p

② CHILD

A child is an element that is directly inside another element.

EXAMPLE

```
<section>
  <p>Text</p>
</section>
```

Here, <p> is the child of <section>.

Example in DOM

h1 is child of body

section is child of body

p, div are children of section

p (second) is child of div

③ DESCENDANT

A descendant is any element inside another element (direct or deep).

EXAMPLE

```
<section>
  <div>
    <p>Text</p>
  </div>
</section>
```

Here, <p> is a descendant of <section>.

Example in DOM

All elements except body are descendants of body. Inside section → p, div, p are all descendants.

④ ANCESTOR

An ancestor is any element above another element (direct or up).

EXAMPLE

```
<body>
  <section>
    <div>
      <p>Text</p>
    </div>
  </section>
</body>
```

Here, <body>, <section> and <div> are ancestors of <p>.

Example in DOM

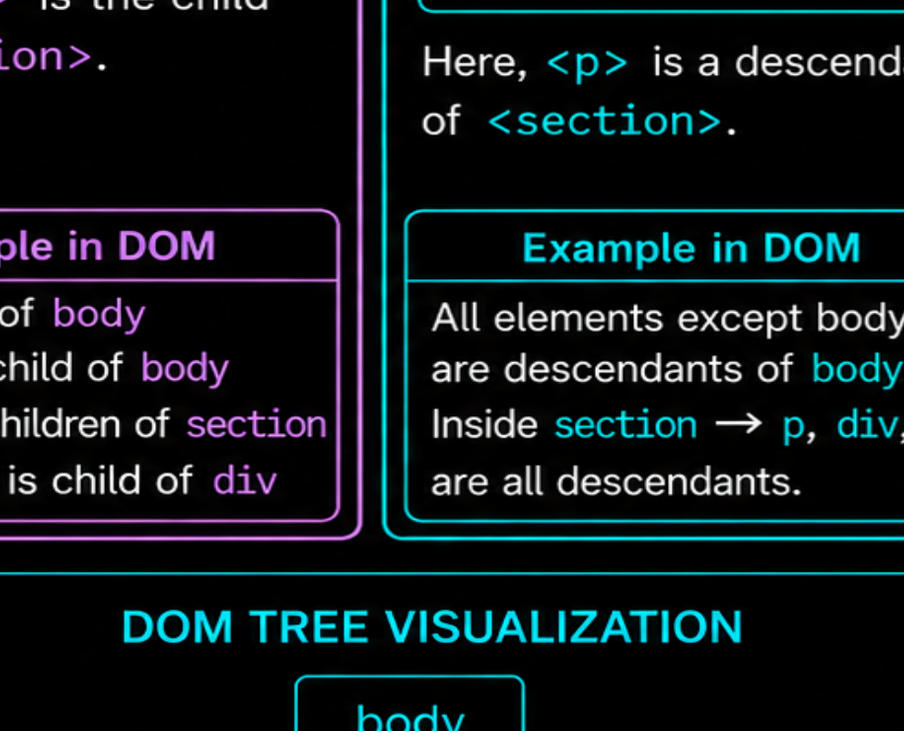
Ancestors of second <p>: div, section, body

Ancestors of div: section, body

GIVEN DOM (HTML)

```
<body>
  <h1>Hello</h1>
  <section>
    <p>First Paragraph</p>
    <div>
      <p>Second Paragraph</p>
    </div>
  </section>
</body>
```

DOM TREE VISUALIZATION



RELATIONSHIP LEGEND

- → ANCESTOR: Any level above
- → PARENT: Directly contains
- → CHILD: Directly inside
- → DESCENDANT: Any level inside

PARENT RELATIONSHIPS

Parent	Child
body	h1
body	section
section	p (First Paragraph)
section	div
div	p (Second Paragraph)

CHILD RELATIONSHIPS

Child	Parent
h1	body
section	body
p (First Paragraph)	section
div	section
p (Second Paragraph)	div

DESCENDANTS

Element	Descendants
body	h1, section, p, div, p
section	p, div, p
div	p (Second Paragraph)
h1	None
p (First Paragraph)	None
p (Second Paragraph)	None

ANCESTORS

Element	Ancestors
h1	body
section	body
p (First Paragraph)	section, body
div	section, body
p (Second Paragraph)	div, section, body

KEY TAKEAWAYS

- Child → direct connection only.
- Descendant → any depth inside.
- Ancestor → any depth above.
- Understanding these relationships is the key to advanced CSS, DOM manipulation, and frontend development.

QUICK REMINDER

PARENT : contains directly
CHILD : inside directly
DESCENDANT : anywhere inside
ANCESTOR : anywhere above

7) Select by Descendant :-

HTML Structure

```
<section>
  <div>
    <p>I am first para</p>
  </div>
  <div>
    <p> I am second para</p>
  </div>
</section>

<section>
  <div>
    <h2>Hello Paji</h2>
  </div>
</section>
```

Descendant Selector

```
section p {
  /* styles */
}
```

Meaning

`section p`

means:
Select every <p> that exists anywhere inside <section > (direct child OR deeply nested child).

Internal Understanding

section
↓
any element inside it
↓
p

So browser checks:

Is <p> somewhere inside <section> ?

If YES → selector matches.

Why Both <p> Elements Get Selected?

Because BOTH paragraphs are descendants of <section>.

Structure:

```
        section
     div       div
      ↓         ↓
      p         p
   (First)   (Second)
```

Example

```
section p {
  color: red;
}
```

targets:
✓ First <p>
✓ Second <p>

Important Point

Descendant selector: A B
does NOT care about direct relationship.
It only checks:
Is B somewhere inside A ?

8) Select by Child :-

Child Selector

```
section > p {
  /* styles */
}
```

Meaning

`section > p`

means:
Select ONLY those <p> elements that are DIRECT children of <section>.

Internal Understanding

section
↓
direct child only
↓
p

Example 1

HTML (Invalid)

```
<body>
  <p>
    <div>Hello Coders</div>
  </p>
</body>
```

CSS

```
p > div {
  color: blue;
}
```

Reason: <div> cannot exist inside <p>.
<p> is meant for paragraph text content only.
Browser automatically fixes invalid HTML.

Why This Is NOT Working?

Because this HTML itself is invalid. Browser internally auto-corrects it.

What Browser Actually Creates

```
<p></p>
<div>Hello Coders</div>
<p></p>
```

Final DOM Browser Creates

body → p, div, p

Now check selector: p > div
Browser asks:
Is <div> direct child of <p> ?
Answer: NO
So selector does not work.

Example 2

HTML (Valid)

```
<body>
  <div>
    <p>Hello Coders</p>
  </div>
</body>
```

CSS

```
div > p {
  color: blue;
}
```

Why This IS Working?

Because structure is valid.
DOM becomes:

body
↓
div
↓
p

Browser checks:
Is <p> direct child of <div> ?
Answer: YES
So selector matches successfully.

Most Important Realization

p can be child of div

BUT

div cannot be child of p

because <p> is meant only for paragraph text content.
Browser automatically fixes invalid HTML structures.

Difference Between Descendant and Child Selector

Descendant Selector (A B)
Selects deep elements also.
Example:
section → div → p
Still works. ✓

Child Selector (A > B)
Selects ONLY direct children.
Example 1:(Works)
section → p
Works. ✓
Example 2:
section → div → p
Does NOT work. ✗

Mental Shortcut

Descendant Selector (A B)
Find B anywhere inside A

Child Selector (A > B)
Find B directly attached to A

BONUS TIP:

For good optimization and faster browser rendering:
Choose smart selectors for better performance and maintainability.

✓ **PREFER SIMPLE AND SPECIFIC SELECTORS**

```
.card
.navbar
.btn-primary
#header
```

✓ **CLASS SELECTORS ARE MOST COMMONLY USED**

They are reusable, flexible, and easy to manage.

✓ **GROUPING SELECTORS ARE COMPLETELY FINE**

`h1, h2, p { }`

✓ **ATTRIBUTE SELECTORS ARE OKAY WHEN REALLY NEEDED**

`input[type="email"]`

✗ **AVOID OVERLY COMPLEX DEEP SELECTORS**

`div > ul > li > a > span`

✗ **AVOID UNNECESSARY UNIVERSAL SELECTOR USAGE**

`* { }`

It matches EVERY element and can increase styling work.

✗ **AVOID EXTREMELY GENERIC SELECTORS IN LARGE APPS**

```
div { }
span { }
```

They affect too many elements and can make styles harder to maintain.

div → div, p, span; div → span; span → a — Too many elements targeted

MODERN BROWSERS ARE HIGHLY OPTIMIZED

You CAN use any selector, but cleaner and simpler selectors improve:

- ✓ Maintainability
- ✓ Scalability
- ✓ Debugging
- ✓ Rendering efficiency

Write selectors that are smart, simple, and sustainable.

CSS COLOUR

① NAMED COLOUR

SYNTAX

```
<style>
  h1 {
    color: blue;
  }
</style>
```

EXAMPLE

`<h1>Hello Coders</h1>`

INTERNALS

Browser has 147 predefined names like: red, green, blue, yellow, orange, purple, black, white, etc.

INTERNAL UNDERSTANDING

color: blue;
↓
Browser looks in its named color table
↓
blue --> (#0000ff)
↓
Applies the actual RGB value (0, 0, 255) to the element

IMPORTANT POINTS

- Easy to remember
- Good for basics
- Limited choices
- Not suitable when exact shade is needed

SOME COMMON NAMED COLOURS

⬛	⬜	🟥	🟩	🟦	🟨	🟧	🟪	■	■	🟫	■
black	white	red	green	blue	yellow	orange	purple	pink	gray	brown	cyan

2) RGB COLOUR

SYNTAX

```
<style>
  p {
    color: rgb(255, 0, 255);
  }
</style>
```

EXAMPLE

`<p>Hello Coders</p>`

INTERNALS

RGB = Red, Green, Blue
Each value can be from 0 to 255
Total possible colors = 256 x 256 x 256
= 16,777,216 (approx 16 million)

HOW RGB WORKS INTERNALLY

rgb(r, g, b)

- r -> red intensity (0-255)
- g -> green intensity (0-255)
- b -> blue intensity (0-255)

Example:
rgb(255, 0, 255)
= Red(255) + Green(0) + Blue(255)
= Magenta

IMPORTANT POINTS

- Very precise control
- Millions of colors
- Best for exact colors
- Used in most designs and themes

RGB VALUES GUIDE

Value	0	128	255
Intensity	None (Off)	Medium	Full (Max)
Example (Red)	rgb(0,0,0)	rgb(128,0,0)	rgb(255,0,0)
Example (Green)	rgb(0,0,0)	rgb(0,128,0)	rgb(0,255,0)
Example (Blue)	rgb(0,0,0)	rgb(0,0,128)	rgb(0,0,255)

COLOR MIXING IN RGB

```
rgb(255, 0, 0)        -> Red
rgb(0, 255, 0)        -> Green
rgb(0, 0, 255)        -> Blue
rgb(255, 255, 0)      -> Yellow    (Red + Green)
rgb(0, 255, 255)      -> Cyan      (Green + Blue)
rgb(255, 0, 255)      -> Magenta   (Red + Blue)
rgb(255, 255, 255)    -> White     (All max)
rgb(0, 0, 0)          -> Black     (All off)
```

③ HEXADECIMAL COLOUR

SYNTAX

```
<style>
  h1 {
    color: #37455d;
  }
</style>
```

EXAMPLE

`<h1>Hello Coders</h1>`

INTERNALS

Hex uses 6 characters: 0-9, a-f, A-F
Format: #RRGGBB
RR -> Red GG -> Green BB -> Blue
Each pair (RR, GG, BB) is hexadecimal
(00 to FF) which is equal to (0 to 255) in decimal.

HOW HEX WORKS INTERNALLY

```
         #37455d
  37        45        5d
  ↓         ↓         ↓
  RR        GG        BB
  ↓         ↓         ↓
  Red      Green     Blue
  ↓         ↓         ↓
  55        69        93 (decimal)
(0-255)   (0-255)   (0-255)
```

IMPORTANT POINTS

- Short, clean & modern
- Easy to copy & use
- Widely used in web development
- Case-insensitive (#abc and #ABC same)

CONVERSION TABLE (HEX <-> DECIMAL)

Hex	00	11	22	33	44	55	66	77	88	99	aa	bb	cc	dd	ee	ff
Decimal	0	17	34	51	68	85	102	119	136	153	170	187	204	221	238	255

QUICK EXAMPLES

```
#000000  -> rgb(0, 0, 0)           (Black)
#ffffff  -> rgb(255, 255, 255)     (White)
#ff0000  -> rgb(255, 0, 0)         (Red)
#00ff00  -> rgb(0, 255, 0)         (Green)
#0000ff  -> rgb(0, 0, 255)         (Blue)
```

④ HSL COLOUR

SYNTAX

```
<style>
  h2 {
    color: hsl(210, 70%, 50%);
  }
</style>
```

EXAMPLE

`<h2>Hello Coders</h2>`

INTERNALS

HSL stands for:
H -> Hue (0 - 360)
S -> Saturation (0% - 100%)
L -> Lightness (0% - 100%)

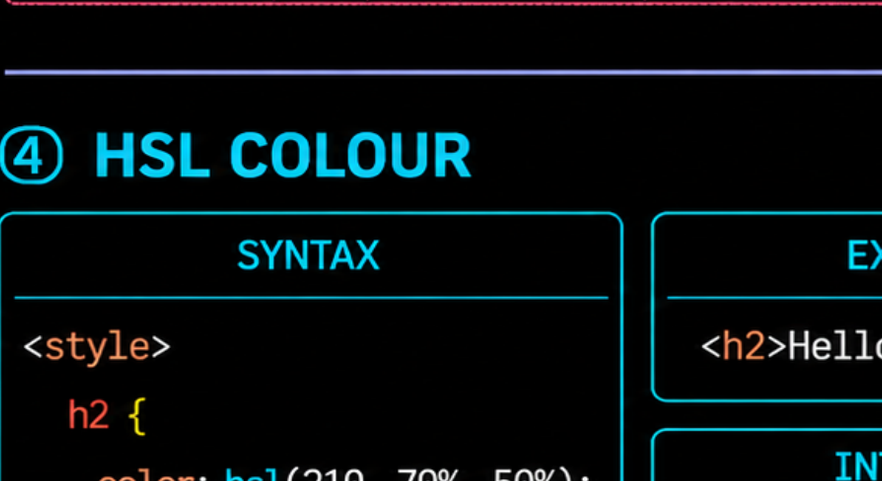
HOW HSL WORKS INTERNALLY

- Hue = actual color (red, blue, green etc.) based on color wheel (0-360°).
- Saturation = intensity of the color. 0% = gray, 100% = full color.
- Lightness = how light or dark the color is. 0% = black, 100% = white.

IMPORTANT POINTS

- Easy to understand
- Perfect for UI/UX
- Great for creating themes, shades, hover effects etc.

COLOR WHEEL (HUE GUIDE)



EXAMPLES OF HSL

```
hsl(0, 100%, 50%)      -> Red
hsl(120, 100%, 50%)    -> Green
hsl(240, 100%, 50%)    -> Blue
hsl(0, 0%, 50%)        -> Gray
hsl(0, 0%, 100%)       -> White
hsl(0, 0%, 0%)         -> Black
```

⑤ ALPHA (TRANSPARENCY) COLOUR

SYNTAX

```
/* Using rgba() */
color: rgba(255, 0, 0, 0.5);

/* Using hsla() */
color: hsla(240, 100%, 50%, 0.3);
```

INTERNALS

Alpha ranges from 0 (fully transparent) to 1 (fully opaque).

EXAMPLE

```
/* rgba (Red with 50% opacity) */
background-color: rgba(255, 0, 0, 0.5);

/* hsla (Blue with 30% opacity) */
background-color: hsla(240, 100%, 50%, 0.3);

/* rgba (Green with 75% opacity) */
background-color: rgba(0, 255, 0, 0.75);
```

HOW IT WORKS INTERNALLY

RGBA Format:

```
  rgba(R,     G,      B,     A)
       ↓      ↓       ↓      ↓
      Red   Green   Blue   Alpha
   (0-255) (0-255) (0-255) (0-1)
```

HSLA Format:

```
  hsla(H,      S,         L,        A)
  Hue    Saturation  Lightness  Alpha
(0-360)   (0-100%)    (0-100%)  (0-1)
```

Alpha (A) controls the transparency:
0 = Fully Transparent
1 = Fully Opaque

IMPORTANT POINTS

- Adds transparency to any colour.
- 0 = invisible
- 1 = fully visible
- Works with both RGB and HSL.
- Useful for overlays, backgrounds, shadows, and hover effects.
- Widely used in modern UI/UX design.

ALPHA VALUES GUIDE (0 TO 1)

	0 (Transparent)	0.25	0.50	0.75	1 (Opaque)
rgba(255, 0, 0, A)	Invisible	25% Visible	50% Visible	75% Visible	Fully Visible
hsla(240, 100%, 50%, A)	Invisible	25% Visible	50% Visible	75% Visible	Fully Visible

QUICK EXAMPLES

```
rgba(255, 0, 0, 0)
/* Transparent Red */

rgba(255, 0, 0, 0.5)
/* 50% Transparent Red */

rgba(0, 255, 0, 0.3)
/* 30% Transparent Green */

hsla(240, 100%, 50%, 0.6)
/* 60% Transparent Blue */

rgba(0, 0, 0, 0.7)
/* 70% Transparent Black */
```